

Day 5 — Hooks & API

Your simple, friendly guide for today

COMING FROM	KnockoutJS / .NET (C#) → moving to React + TypeScript
TODAY'S GOAL	Load real data from a server with useEffect — and handle loading, errors, pagination and search (using the DummyJSON API).

How to use this: Read **Section 1 (Learning Plan)** for each idea with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make it click. Keep **Section 3 (Common Mistakes)** handy while you practise, and copy **Section 4 (Concept Notes)** by pen for recall & interviews.

1 Learning Plan

On Day 4 you learned `useState` and how to keep data inside a component. But that data was made up by you (hard-coded). Today you learn how to **go get real data from a server** — and how to handle the waiting, the errors, and the empty results while you do it.

Think of it like this: in .NET you call an API with `HttpClient`, `await` the response, and turn the JSON into objects. React is the same idea. The only new part is telling React *"run this fetch at the right time, and clean up after yourself."* That "right time + clean up" job belongs to a hook called `useEffect`.

We will build toward one small screen: a **Users list** that loads from the free

`https://dummyjson.com/users` API, shows a **loading spinner**, shows an **error message** if something breaks, has a **search box**, and has **Prev / Next** paging buttons.

New word — **API**: a URL on a server you can call to get or send data (here, a list of users). **Hook**: a special React function whose name starts with `use` that lets a component "hook into" React features (state, effects, etc.).

Part A — Core Topics

1. `useEffect` (with dependency array + cleanup)

A component's job is to return UI. But sometimes you also need to do a **side effect** — something *outside* just returning UI, like fetching data, starting a timer, or subscribing to something. `useEffect` runs that code **after** React has drawn the screen. The second argument, the **dependency array** `[]`, tells React *when* to re-run: `[]` = run once after the first render; `[query]` = run again whenever `query` changes. If your effect returns a function, React runs that function to **clean up** (stop the timer, cancel the request) before the next run and when the component leaves the screen.

```
import { useEffect, useState } from "react";

function Clock() {
  const [now, setNow] = useState<Date>(new Date());

  useEffect(() => {
    const id = setInterval(() => setNow(new Date()), 1000);
    return () => clearInterval(id); // cleanup: runs before re-run and on unmount
  }, []); // [] = run once after the first render

  return <p>{now.toLocaleTimeString()}</p>;
}
```

- **Coming from .NET / KnockoutJS:** the cleanup function is like `IDisposable` / `Dispose()` or disposing a KnockoutJS subscription — you free what you started.
- **You'll be able to...** run code after render (like fetching) and clean it up safely, with no leftover timers or requests.

2. Fetching with the built-in `fetch`

`fetch` is built into every browser — no install needed. It returns a `Promise` (Day 1 / C# `Task`), so you `await` it. Two gotchas: (1) `fetch` **does not throw on HTTP errors** like 404 or 500 — you must check `res.ok` yourself; (2) the body is not JSON yet — you call `await res.json()` to parse it. You usually put the `fetch` inside a `useEffect`.

```
import { useEffect, useState } from "react";

interface User { id: number; firstName: string; }
interface UsersResponse { users: User[]; total: number; }

function UsersList() {
  const [users, setUsers] = useState<User[]>([]);

  useEffect(() => {
    async function load() {
      const res = await fetch("https://dummyjson.com/users");
      if (!res.ok) throw new Error(`HTTP ${res.status}`); // fetch won't throw on 404/500
      const data = (await res.json()) as UsersResponse; // parse the JSON body
      setUsers(data.users);
    }
    void load(); // (we add try/catch for errors in topic 4)
  }, []);

  return <ul>{users.map((u) => <li key={u.id}>{u.firstName}</li>)}</ul>;
}
```

Note: DummyJSON wraps the list under a key: `{ users, total, skip, limit }`. On Day 2 you made a generic `ApiResponse<T>`; here the key is named `users`, so we just type the exact shape as `UsersResponse`. Keep it simple.

- **Coming from .NET:** this is `HttpClient.GetAsync( ... )`, then checking `response.IsSuccessStatusCode`, then deserializing the JSON yourself.
- **You'll be able to...** load real data from any API using only the browser — zero libraries.

3. Fetching with axios

`axios` is a small library that makes API calls nicer. It **parses the JSON for you** (`res.data` is already an object), and it **throws automatically** on HTTP errors (4xx/5xx), so no manual `res.ok` check. With TypeScript you tell it the shape with `axios.get<T>()`.

```
import axios from "axios";

interface UsersResponse {
  users: { id: number; firstName: string }[];
  total: number;
}

async function loadUsers(): Promise<UsersResponse> {
  const res = await axios.get<UsersResponse>("https://dummyjson.com/users");
  return res.data; // already parsed; axios throws on 4xx/5xx for you
}
```

- **Coming from .NET:** axios is like a preconfigured `HttpClient` that deserializes the JSON and throws on a bad status code — less plumbing than raw `fetch`.
- **You'll be able to...** call APIs with less boilerplate and get automatic error throwing + typed data.

4. Loading / error / empty states

A real screen has more than "data". While the request is running the user should see **loading**; if it fails they should see a friendly **error**; if it succeeds but there is nothing, they should see an **empty** message ("No users found"). A clean trick is to keep one `status` string (a small **discriminated union** from Day 2) instead of three separate booleans.

```
type Status = "loading" | "error" | "ready";

function View({ status, count }: { status: Status; count: number }) {
  if (status === "loading") return <p>Loading ... </p>;           // swap for a spinner
  if (status === "error") return <p>Something went wrong. Please try again.</p>;
  if (count === 0) return <p>No users found.</p>;               // empty state
  return <p>{count} users</p>;
}
```

- **Coming from KnockoutJS:** instead of juggling `isLoading()`, `hasError()` observables, you keep one `status` value — it can never be "loading AND error" at once.
 - **You'll be able to...** always show the user a clear state, never a blank, frozen screen.
-

5. Environment variables in Vite

An **environment variable** is a setting that lives *outside* your code, so you can use a different value for dev vs production (like the API URL). Vite reads these from a `.env` file. **Important rule:** the name must start with `VITE_`, and you read it with `import.meta.env.VITE_...`. (See the "right way" extra below for the `.env` file and the type declaration.)

```
// vite-env.d.ts declares VITE_API_URL as a string (see extras below)
const baseUrl = import.meta.env.VITE_API_URL; // e.g. "https://dummyjson.com"
fetch(`${baseUrl}/users`); // build the URL from the env variable
```

- **Coming from .NET:** this is like `appsettings.json` + `IConfiguration` — but see the warning below: front-end env values are **public**, unlike a server-side secret.
- **You'll be able to...** change the API URL per environment (dev / staging / prod) without editing code.

6. Pagination

APIs don't send 10,000 rows at once — they send one **page** at a time. DummyJSON uses two query params: `limit` (how many per page) and `skip` (how many to jump over). Page 1 = `skip=0`, page 2 = `skip=10`, and so on. You keep `skip` in state and change it with Prev / Next buttons.

```
import { useState } from "react";

function Pager() {
  const [skip, setSkip] = useState<number>(0);
  const limit = 10;
  // fetch(`https://dummyjson.com/users?limit=${limit}&skip=${skip}`)
  return (
    <div>
      <button onClick={() => setSkip((s) => Math.max(0, s - limit))}>Prev</button>
      <span> skip: {skip} </span>
      <button onClick={() => setSkip((s) => s + limit)}>Next</button>
    </div>
  );
}
```

- **Coming from .NET / LINQ:** `skip` and `limit` are exactly `.Skip(n).Take(m)` in LINQ / EF — same idea, sent over the URL.
- **You'll be able to...** show large lists page-by-page instead of dumping everything at once.

7. Search

DummyJSON has a search endpoint: `GET /users/search?q=John`. It returns the same shape (`{ users, total, skip, limit }`). You keep the typed text in state (a **controlled input** from Day 4) and, when it changes, re-fetch from the search URL. The **server** does the filtering, so you don't download everyone and filter in the browser.

```
import { useState } from "react";

function Search({ onSearch }: { onSearch: (q: string) => void }) {
  const [query, setQuery] = useState<string>("");
  return (
    <input
      value={query}
      placeholder="Search users ... "
      onChange={(e) => {
        setQuery(e.target.value);
        onSearch(e.target.value); // tell the parent to re-fetch
      }}
    />
  );
}
```

- **Coming from .NET:** the server runs the `Where( ... )` on its side; you just pass `q`. The client stays light.
- **You'll be able to...** let users filter the list live, straight from the server.

Putting it all together — the Users screen

This one component uses **every** piece above: `useEffect` with a dependency array `[query, skip]`, `fetch` with a manual `res.ok` check, loading / error / empty states, search, Prev / Next pagination, and an `AbortController` (explained in the extras) so old requests get cancelled.

```

import { useEffect, useState } from "react";

interface User {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
}

interface UsersResponse {
  users: User[];
  total: number;
  skip: number;
  limit: number;
}

const BASE_URL = "https://dummyjson.com"; // later: import.meta.env.VITE_API_URL

export default function UsersScreen() {
  const [users, setUsers] = useState<User[]>([]);
  const [total, setTotal] = useState<number>(0);
  const [status, setStatus] = useState<"loading" | "error" | "ready">("loading");
  const [query, setQuery] = useState<string>("");
  const [skip, setSkip] = useState<number>(0);
  const limit = 10;

  useEffect(() => {
    const controller = new AbortController();

    async function load() {
      setStatus("loading");
      try {
        const url = query
          ? `${BASE_URL}/users/search?q=${query}&limit=${limit}&skip=${skip}`
          : `${BASE_URL}/users?limit=${limit}&skip=${skip}`;
        const res = await fetch(url, { signal: controller.signal });
        if (!res.ok) throw new Error(`HTTP ${res.status}`);
        const data = (await res.json()) as UsersResponse;
        setUsers(data.users);
        setTotal(data.total);
        setStatus("ready");
      } catch (err) {

```

```

        // if we cancelled the request on purpose, do nothing
        if (err instanceof DOMException && err.name === "AbortError") return;
        setStatus("error");
    }
}

void load();
return () => controller.abort(); // cleanup: cancel the old request
}, [query, skip]); // re-run when the search text OR the page changes

return (
    <div>
        <input
            value={query}
            placeholder="Search users ... "
            onChange={(e) => {
                setSkip(0); // new search → jump back to the first page
                setQuery(e.target.value);
            }}
        />

        {status === "loading" && <p>Loading ... </p>}
        {status === "error" && <p>Something went wrong. Please try again.</p>}
        {status === "ready" && users.length === 0 && <p>No users found.</p>}

        {status === "ready" && users.length > 0 && (
            <
                <ul>
                    {users.map((u) => (
                        <li key={u.id}>
                            {u.firstName} {u.lastName} - {u.email}
                        </li>
                    ))}
                </ul>
                <button disabled={skip === 0} onClick={() => setSkip((s) => Math.max(0, s -
limit))}>
                    Prev
                </button>
                <button disabled={skip + limit ≥ total} onClick={() => setSkip((s) => s +
limit)}>
                    Next
                </button>
            >

```

```

    </>
  })
</div>
);
}

```

Part B — Extra Topics

fetch vs axios

	<code>fetch</code> (built-in)	<code>axios</code> (library)
Install needed?	No	Yes (<code>npm i axios</code>)
Parse JSON	You call <code>await res.json()</code>	Done for you — use <code>res.data</code>
HTTP errors (404/500)	Does not throw — check <code>res.ok</code>	Throws automatically
Base URL / headers	You repeat them each call	Set once (see below)

The big axios win is a shared instance — set the base URL and common headers once, and even add an **interceptor** (a function that runs on every request/response, e.g. to attach a token):

```

import axios from "axios";

const api = axios.create({ baseURL: "https://dummyjson.com" });
api.interceptors.request.use((config) => config); // e.g. attach an auth header here
// now just: api.get("/users") – no repeating the base URL

```

Rule of thumb: `fetch` is fine for small apps; reach for axios when you have many calls or shared config.

API error-handling best practices

- With `fetch`, **always check** `res.ok` (or `try/catch` with axios) — never assume success.
- Show a **friendly message** ("Something went wrong. Please try again."). Never dump the raw error object or stack trace onto the screen — it confuses users and can leak internal details.
- Handle the **empty result** separately from the error. "No users found" is a *success* with zero rows, not a failure.
- Log the real error to the console (`console.error(err)`) for you, but show the friendly text to the user.
- Keep one clear `status` (`loading` / `error` / `ready`) so the UI can never contradict itself.

Environment variables the RIGHT way in Vite

Two files. First the `.env` at the project root (variable name **must** start with `VITE_`), then a type declaration so TypeScript knows it's a `string`:

```
// .env (project root)
// VITE_API_URL=https://dummyjson.com

// src/vite-env.d.ts (so import.meta.env.VITE_API_URL is typed as string)
interface ImportMetaEnv {
  readonly VITE_API_URL: string;
}
interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

- Read it with `import.meta.env.VITE_API_URL`.
- **Coming from Create React App:** CRA used `process.env.REACT_APP_XXX`; Vite uses `import.meta.env.VITE_XXX`. Different prefix, different object.
- ⚠️ **WARNING — front-end env vars are PUBLIC.** Anything you put in a `VITE_` variable is **baked into the JavaScript that ships to the browser**. Anyone can open DevTools and read it. So `VITE_API_URL` is fine, but **never** put API keys, passwords, or secrets there. Real secrets stay on a backend server (like a .NET API), not in the front-end.

AbortController / cleanup (cancel a fetch)

When the user types fast or leaves the page, old requests can still be "in flight". If a slow old request finishes *after* a newer one, it can overwrite good data (a **race condition**), or try to set state on a component that's already gone (React warns about this). An `AbortController` lets you cancel a request. Start one in the effect, pass its `signal` to `fetch`, and `abort()` it in the cleanup.

```
import { useEffect } from "react";

function Example() {
  useEffect(() => {
    const controller = new AbortController();
    void fetch("https://dummyjson.com/users", { signal: controller.signal });
    return () => controller.abort(); // cancel on unmount or when deps change
  }, []);
  return null;
}
```

An aborted `fetch` rejects with a `DOMException` named `"AbortError"` — catch it and ignore it (that's the pattern in the full Users screen above). This kills both problems: no race conditions, and no "set state

after unmount".

React StrictMode runs effects twice (on purpose)

In development, React wraps your app in `<StrictMode>`. It **mounts, unmounts, and remounts** each component once — so your `useEffect` (and its cleanup) runs **twice** on the first load. This is a feature, not a bug: it's React checking that your cleanup actually works. It **only happens in dev**, never in the production build.

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";

function App() {
  return <h1>Users</h1>;
}

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

If you see your fetch fire twice in dev, don't panic — that's StrictMode. And because you wrote proper cleanup (the `AbortController` above), the doubled run is harmless. If it *does* cause a visible bug, that's a hint your cleanup is missing something.

Heads-up: this becomes a custom hook on Day 7

Right now the fetch logic lives inside the component. On **Day 7** we'll lift it into a reusable **custom hook** — a plain function like `function useUsers(query: string, skip: number)` that returns `{ users, status, total }`. That way any screen can fetch users in one line. Deep custom hooks and reusable components are a Day 7 topic, so just notice the pattern for now.

Recap: `useEffect` decides *when* your side effects run and cleans them up. `fetch` / `axios` get the data. `status` + empty checks keep the UI honest. `VITE_` env vars keep the URL configurable (but public). `skip` / `limit` page it, `q` searches it, and `AbortController` cancels stale requests. Put those together and you have the Users screen above.

2

Detailed Notes — Read Once, Understand Everything

The big idea first (read this slowly)

A **side effect** is any work your component does that reaches **outside** React.

Think of React's job as one simple thing: take your state and props, and return the JSX that should be on screen. That's a pure calculation. But real apps also need to do *other* stuff — call an API, read the clock, set a timer, save to `localStorage`, talk to the browser. All of that reaches outside React. That "reaching outside" work is a **side effect**.

Calling an API is the classic side effect. In C#/.NET you'd reach out with `HttpClient`. In React the reaching-out is the same idea — but *where* you put it matters.

Here's the rule: **side effects don't belong in the render**. They belong in `useEffect`, which runs your effect code **after** React has drawn the screen — never during. That one sentence is the whole day. Everything below just fills it in.

1. What a side effect is, and why you can't just fetch in the component body

Your component body runs **every single time** the component renders. And React can render a component many times (whenever state or props change). So the body must stay clean: just calculate and return JSX.

If you call `fetch` directly in the body, two bad things happen:

1. It fires on **every** render. State changes → re-render → fetch → set state → re-render → fetch → ... an endless loop.
2. `fetch` is async, so the data isn't ready when render returns anyway. You can't `await` in the body.

```
function UserList() {  
  // ❌ DON'T do this - runs on every render, causes an infinite loop  
  // const res = await fetch("https://dummyjson.com/users");  
  // ...  
  return <div />;  
}
```

So we need a place that runs *after* render and only when we *want* it to. That place is `useEffect`. (If you've touched Blazor, this is a bit like `OnInitializedAsync` — code that runs once the component is ready, not while it's being built. WPF folks: think of the `Loaded` event, not the constructor.)

2. `useEffect` basics — the effect function and the dependency array

`useEffect` takes **two arguments**:

1. A **function** — the effect. This is the side-effect work you want to run.
2. A **dependency array** — a list of values React watches.

```
import { useEffect } from "react";

useEffect(() => {
  console.log("This runs after the screen is painted");
}, []); // ← the dependency array
```

When does the effect run?

- Always after the **first** render (once the component is on screen).
- Again after **any later render where a value in the array changed**.

The dependency array is how you say "only re-run when *these* things change":

- `[]` (empty) → run **once**, after the first render. Never again. Perfect for "load the data when the page opens."
- `[page]` → run after first render, and again **every time** `page` changes.
- **No array at all** → run after **every** render. You almost never want this — it's the infinite-loop trap again.

```
useEffect(() => {
  loadUsers();
}, [page]); // re-runs whenever `page` changes
```

Rule of thumb: **every value from inside the component that you use inside the effect should be listed in the array**. The ESLint rule `react-hooks/exhaustive-deps` will nag you if you forget one — trust it.

3. The cleanup function — return a function from your effect

Sometimes an effect starts something that needs to be **stopped** later: a timer, a subscription, an open network request. To handle that, your effect can **return a function**. React calls that returned function to "clean up."

```
useEffect(() => {
  const id = setInterval(() => console.log("tick"), 1000);

  // cleanup: React runs this to stop the timer
  return () => {
    clearInterval(id);
  };
}, []);
```

When does cleanup run? Two moments:

1. **Before the effect runs again** (because a dependency changed). React cleans up the *old* effect first, then runs the new one.
2. **When the component unmounts** (leaves the screen for good).

So the pattern is: *start* on the way in, *stop* on the way out. If you never stop things (timers, requests), you get bugs and memory leaks. Cleanup is your "undo."

4. Fetching with `fetch` — and the one gotcha that trips everyone

`fetch` is the browser's built-in way to make HTTP calls (like `HttpClient` in .NET). It's async, so we write an **async helper inside** the effect and call it. (You can't make the effect function itself `async` — an async function returns a Promise, but React expects the effect to return either nothing or a cleanup function.)

```
useEffect(() => {
  async function loadUsers() {
    const res = await fetch("https://dummyjson.com/users");
    const data = await res.json(); // parse the JSON body
    console.log(data);
  }
  void loadUsers(); // `void` just says "I'm not awaiting this on purpose"
}, []);
```

Two steps every time: `await fetch(...)` gets the response, then `await res.json()` reads and parses the body into an object.

The big gotcha: `fetch` does **NOT** throw on a 404 or 500. In .NET, a bad status code is easy to catch — or you call `EnsureSuccessStatusCode()`. With `fetch`, a 404 is still a "successful" promise. It only rejects on a *network* failure (no internet, DNS dead). So **you** must check the status yourself with `res.ok` (true for 200–299):

```
const res = await fetch("https://dummyjson.com/users");
if (!res.ok) {
  throw new Error(`Request failed: ${res.status}`); // your own EnsureSuccessStatusCode
}
const data = await res.json();
```

Forget this and a 404 will happily try to parse an error page as your data. Always check `res.ok`.

5. The loading / error / data pattern (three pieces of state)

A real fetch has **three possible situations** on screen, and you should handle all three:

- **Loading** — the request is in flight. Show a spinner or "Loading...".
- **Error** — something went wrong. Show a friendly message.
- **Data** — it worked. Show the list (and handle the **empty** case: data arrived but the list is empty).

The simplest version keeps three pieces of state:

```

const [users, setUsers] = useState<User[]>([]);
const [loading, setLoading] = useState<boolean>(true);
const [error, setError] = useState<string | null>(null);

useEffect(() => {
  async function loadUsers() {
    try {
      setLoading(true);
      setError(null);
      const res = await fetch("https://dummyjson.com/users");
      if (!res.ok) throw new Error(`Request failed: ${res.status}`);
      const data: UsersResponse = await res.json();
      setUsers(data.users);
    } catch (err) {
      setError(err instanceof Error ? err.message : "Something went wrong");
    } finally {
      setLoading(false); // runs whether we succeeded or failed
    }
  }
  void loadUsers();
}, []);

if (loading) return <p>Loading...</p>;
if (error) return <p>Error: {error}</p>;
if (users.length === 0) return <p>No users found.</p>;

return (
  <ul>
    {users.map((u) => (
      <li key={u.id}>{u.firstName} {u.lastName}</li>
    ))}
  </ul>
);

```

Notice the order of the `if` guards: loading first, then error, then empty, then the happy path. Read top to bottom, exactly one wins.

Tie-in to Day 2: three separate booleans can drift into "impossible" combos (loading *and* error at once). The cleaner, type-safe way is the **discriminated union** you learned on Day 2 — one status field that can only be one thing at a time:

```
type UsersState =  
  | { status: "loading" }  
  | { status: "error"; message: string }  
  | { status: "success"; users: User[] };
```

Now `loading` + `error` together is *impossible by design*, and TypeScript narrows the type for you inside each branch. Start with the three-booleans version to understand it; reach for the union when you want it airtight.

6. Typing the response

Never leave API data as `any`. Describe the shape with an `interface` so TypeScript checks every property. DummyJSON's `/users` endpoint returns this shape:

```
interface User {  
  id: number;  
  firstName: string;  
  lastName: string;  
  email: string;  
}  
  
interface UsersResponse {  
  users: User[];  
  total: number; // how many users exist in total  
  skip: number; // how many were skipped  
  limit: number; // how many were returned this call  
}
```

Then annotate the parsed data: `const data: UsersResponse = await res.json();`. Now `data.users` is a typed `User[]`, and a typo like `data.userz` is a compile error, not a 2am surprise. This is the same "type the boundary" habit from Day 2 — the network is the edge of your app, so type it there.

7. Fetching with `axios` — a friendlier client

`axios` is a popular library (install it: `npm install axios`). It does the annoying parts for you, so many teams prefer it over raw `fetch`. Two big conveniences:

1. **Auto-JSON.** No `res.json()` step — the parsed body is already on `response.data`.
2. **It throws on non-2xx.** A 404 or 500 rejects the promise, so your `catch` runs automatically. This is the `EnsureSuccessStatusCode()` behaviour you missed with `fetch` — you get it for free.

```
import axios from "axios";

const res = await axios.get<UsersResponse>("https://dummyjson.com/users");
const data = res.data; // already parsed AND typed as UsersResponse – no res.ok, no
res.json()
setUsers(data.users);
```

Even nicer: make **one shared instance** with a `baseUrl` (and headers, timeouts, etc.), then call short paths everywhere. This is very close to configuring a single `HttpClient` in .NET and reusing it:

```
// api.ts
import axios from "axios";

export const api = axios.create({
  baseUrl: "https://dummyjson.com",
});

// anywhere else:
const res = await api.get<UsersResponse>("/users?limit=10");
```

`fetch` vs `axios`: `fetch` is built in, zero dependencies, but you check `res.ok` and call `res.json()` yourself. `axios` is an extra package but removes that boilerplate and throws on bad statuses. Both are fine — know both.

8. Environment variables — and why they are PUBLIC

You don't want the API base URL hard-coded in ten files. Put it in an **environment variable** instead. With Vite (your build tool), you make a `.env` file at the project root:

```
# .env
VITE_API_URL=https://dummyjson.com
```

Read it in code through `import.meta.env`:

```
const baseUrl = import.meta.env.VITE_API_URL;
```

To keep TypeScript happy, declare the type once (usually in `vite-env.d.ts`):

```
interface ImportMetaEnv {
  readonly VITE_API_URL: string;
}
interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

Two rules you must remember:

1. **The `VITE_` prefix is required.** Vite only exposes variables that start with `VITE_`. `API_URL=...` (no prefix) will be `undefined` in the browser — this is on purpose.
2. **These variables are PUBLIC.** Vite bakes them into the JavaScript bundle that ships to the browser. Anyone can open DevTools and read them. So **never** put secrets here — no passwords, no private API keys, no DB strings. This is the opposite of .NET's `appsettings.json` on a server, which stays private. Frontend env vars are just for non-secret config like URLs and feature flags. Secrets live on a backend.

9. Race conditions — and how cleanup fixes them

Here's a sneaky bug. Imagine a search box. The user types fast:

1. They type "a" → you fire request A.
2. They type "ab" → you fire request B.
3. Request **B** comes back first (networks are unpredictable).
4. Request **A** comes back *after* — and overwrites the screen with results for "a".

Now the box says "ab" but the list shows "a" results. That's a **race condition**: two requests racing, and the slow-but-older one wins. Out-of-order results.

The fix is **cleanup**. When the effect re-runs (or the component unmounts), cancel or ignore the old request. Two common ways:

Way A — `AbortController` (the proper cancel; this is basically React's `CancellationToken`):

```

useEffect(() => {
  const controller = new AbortController();

  async function loadUsers() {
    try {
      const res = await fetch("https://dummyjson.com/users", {
        signal: controller.signal, // hook the request up to the canceller
      });
      if (!res.ok) throw new Error(`Request failed: ${res.status}`);
      const data: UsersResponse = await res.json();
      setUsers(data.users);
    } catch (err) {
      // an aborted request throws - that's expected, so ignore it
      if (err instanceof DOMException && err.name === "AbortError") return;
      setError(err instanceof Error ? err.message : "Something went wrong");
    }
  }

  void loadUsers();

  return () => {
    controller.abort(); // cancel the in-flight request on cleanup
  };
}, [query]);

```

Way B — an "ignore" flag (simpler; doesn't cancel the network call, just ignores its result):

```

useEffect(() => {
  let ignore = false;

  async function loadUsers() {
    const res = await fetch("https://dummyjson.com/users");
    const data: UsersResponse = await res.json();
    if (!ignore) setUsers(data.users); // only apply if still the latest effect
  }

  void loadUsers();

  return () => {
    ignore = true; // mark the old effect as stale
  };
}, [query]);

```

Both work. `AbortController` is better because it also stops wasted network work. The flag is easier to read. Either way, the lesson is the same: **when an effect can be replaced, clean up the old one.**

10. Pagination with `limit` + `skip`

DummyJSON pages with two query params: `limit` (how many to fetch) and `skip` (how many to jump over). So page 0 is `skip=0`, page 1 is `skip=10`, page 2 is `skip=20`, and so on — `skip = page * limit`.

Keep the current page in state. Put `page` in the dependency array so the effect **refetches whenever the page changes**:

```

const [page, setPage] = useState<number>(0);
const [total, setTotal] = useState<number>(0);
const limit = 10;

useEffect(() => {
  async function loadUsers() {
    const skip = page * limit;
    const res = await fetch(
      `https://dummyjson.com/users?limit=${limit}&skip=${skip}`,
    );
    const data: UsersResponse = await res.json();
    setUsers(data.users);
    setTotal(data.total); // total tells us when to stop
  }
  void loadUsers();
}, [page]);

```

The Prev/Next buttons just change `page` — they don't fetch directly. Changing state triggers the re-render, the dependency changes, and the effect refetches. Disable the buttons at the edges so you can't page past the ends:

```

<button onClick={() => setPage((p) => Math.max(0, p - 1))} disabled={page === 0}>
  Prev
</button>
<button
  onClick={() => setPage((p) => p + 1)}
  disabled={(page + 1) * limit ≥ total}
>
  Next
</button>

```

This is the key mental shift from KnockoutJS: you don't imperatively "go fetch now." You change state, and the effect reacts. State is the trigger.

11. Search — with a light touch of debounce

Search is the same idea: keep the query in state, and refetch when it changes. DummyJSON has `/users/search?q=...`.

But if you fetch on *every keystroke*, typing "John" fires four requests (J, Jo, Joh, John) — wasteful and race-prone. The fix is **debounce**: wait until the user *stops* typing for a moment (say 400ms), then fetch once.

The trick is two effects. The first waits and copies `query` into `debounced` (cancelling the wait via cleanup every keystroke). The second fetches only when `debounced` settles:

```
const [query, setQuery] = useState<string>("");
const [debounced, setDebounced] = useState<string>("");

// wait 400ms after the last keystroke, then update `debounced`
useEffect(() => {
  const id = setTimeout(() => setDebounced(query), 400);
  return () => clearTimeout(id); // typing again cancels the previous wait
}, [query]);

// fetch only when the debounced value settles
useEffect(() => {
  async function search() {
    const url = debounced
      ? `https://dummyjson.com/users/search?q=${debounced}`
      : "https://dummyjson.com/users";
    const res = await fetch(url);
    const data: UsersResponse = await res.json();
    setUsers(data.users);
  }
  void search();
}, [debounced]);
```

See how cleanup (`clearTimeout`) is doing the heavy lifting again? Every new keystroke cancels the pending timer, so only the *last* pause actually fires. (Deeper debounce and packaging this into a reusable hook is a Day 7 topic — for now, just get the idea.)

12. StrictMode double-runs — don't panic

In development, if your app is wrapped in `<React.StrictMode>` (Vite adds this by default), React **runs each effect twice** on mount: it mounts, runs the effect, runs the cleanup, then mounts and runs the effect again. You'll see two log lines, or two network calls, and think you broke something.

You didn't. This is intentional. React does it to *stress-test* your cleanup — if your effect leaks or misbehaves when run twice, StrictMode surfaces it now instead of in production. If your effect has proper cleanup (like sections 9–11), the double-run is harmless.

This only happens in **dev**. In the production build it runs once. So: if you see two fetches locally, that's the feature working, not a bug.

A quick note on custom hooks: you'll notice all these effects look similar. You *can* wrap them into your own reusable hook like `useUsers()` — and that's great. But building custom hooks properly (and reusable components) is **Day 7**, so we'll only wave at it here. For now, keep the logic inside the component so the pattern stays visible.

One-line summary

Side effects (like API calls) go in `useEffect` — it runs your effect *after* render, re-runs when its **dependencies** change, and **cleans up** before the next run or on unmount; use loading/error/data state, type the response, prefer `res.ok` / `AbortController` with `fetch` (or let `axios` throw for you), keep secrets out of `VITE_` env vars, and drive pagination and search purely by changing state.

3 Common Mistakes to Avoid

Data fetching is where beginners trip most. Each trap has a quick ✖ / ✔. Examples share this setup:

```
import { useEffect, useState } from "react";

interface User { id: number; firstName: string; lastName: string; }
// DummyJSON wraps the list: { users, total, skip, limit }
interface UsersResponse { users: User[]; total: number; skip: number; limit: number; }
```

1. Forgetting the dependency array → runs after EVERY render

No second argument = the effect runs after every render. `setState` inside → re-render → runs again → fetches again → forever. The #1 cause of an app that hammers the API.

```
// ✖ no [] → setUsers re-renders → runs again ...
useEffect(() => {
  fetch("https://dummyjson.com/users")
    .then((res) => res.json())
    .then((data: UsersResponse) => setUsers(data.users));
});

// ✔ [] = run once on mount
useEffect(() => {
  fetch("https://dummyjson.com/users")
    .then((res) => res.json())
    .then((data: UsersResponse) => setUsers(data.users));
}, []);
```

`[]` = once. `[page]` = again when `page` changes. No array = every render (almost never right).

2. A value the effect sets is also in the deps → infinite loop

List a value in deps *and* change it inside → loop. Depend on the *inputs* (page, search text), never the data you fetch.

```
// ❌ sets users, and users is in deps → forever
useEffect(() => {
  fetch("https://dummyjson.com/users")
    .then((res) => res.json())
    .then((data: UsersResponse) => setUsers(data.users));
}, [users]);

// ✅ depend on page instead
const [page, setPage] = useState(0);
useEffect(() => {
  const skip = page * 10;
  fetch(`https://dummyjson.com/users?limit=10&skip=${skip}`)
    .then((res) => res.json())
    .then((data: UsersResponse) => setUsers(data.users));
}, [page]);
```

3. Making the effect callback itself `async` → wrong shape

`async` returns a `Promise`, but `useEffect` wants nothing or a cleanup function. So `useEffect(async () => ...)` is a TypeScript error and breaks cleanup.

```
// ❌ won't compile (kept as a comment on purpose):
// useEffect(async () => {
//   const res = await fetch("https://dummyjson.com/users");
//   setUsers(((await res.json()) as UsersResponse).users);
// }, []);

// ✅ define an async function inside, then call it
useEffect(() => {
  async function loadUsers() {
    const res = await fetch("https://dummyjson.com/users");
    const data: UsersResponse = await res.json();
    setUsers(data.users);
  }
  loadUsers(); // effect itself stays sync
}, []);
```

You keep your familiar `async` / `await` (same as C#); the effect gets the plain function it wants.

4. Assuming `fetch` throws on 404 / 500 → it doesn't

Unlike `HttpClient`, `fetch` does **not** reject on 404/500 — only on network failures (no internet, DNS, CORS). A 404 is still a "successful" promise. Check `res.ok` yourself — it's the manual

`EnsureSuccessStatusCode()` .

```
// ❌ 404 body isn't the shape you expect
const bad = await (await fetch("https://dummyjson.com/users/999999")).json();

// ✅ check res.ok and throw
const res = await fetch("https://dummyjson.com/users/999999");
if (!res.ok) throw new Error(`Request failed with status ${res.status}`);
const data: UsersResponse = await res.json();
```

axios differs: it throws automatically on 4xx/5xx, so just use `try/catch` — no `res.ok` check.

5. No loading or error handling → blank screen or crash

Track three extra things: **loading**, **error**, and the **empty** case (worked, but zero results).

```

const [users, setUsers] = useState<User[]>([]);
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState<string | null>(null);

useEffect(() => {
  async function loadUsers() {
    setIsLoading(true);
    setError(null);
    try {
      const res = await fetch("https://dummyjson.com/users");
      if (!res.ok) throw new Error(`Failed: ${res.status}`);
      const data: UsersResponse = await res.json();
      setUsers(data.users);
    } catch {
      setError("Could not load users. Please try again.");
    } finally {
      setIsLoading(false); // runs on success OR failure
    }
  }
  loadUsers();
}, []);

// JSX covers all four:
// if (isLoading) return <p>Loading...</p>;
// if (error) return <p>{error}</p>;
// if (users.length === 0) return <p>No users found.</p>;
// return <ul>...</ul>;

```

finally is your friend — same as C#, it always runs, so loading never gets stuck on.

6. Not cancelling old requests → race conditions & stale results

Fast typing fires many requests that return out of order, so a slow old one overwrites the newest → wrong list (plus React's "can't set state on an unmounted component" warning). **AbortController** is the browser's **CancellationToken**.

```
// ✅ cancel the previous request in cleanup
useEffect(() => {
  const controller = new AbortController();
  async function search() {
    try {
      const res = await fetch(
        `https://dummyjson.com/users/search?q=${encodeURIComponent(query)}`,
        { signal: controller.signal }
      );
      if (!res.ok) throw new Error(`Failed: ${res.status}`);
      const data: UsersResponse = await res.json();
      setUsers(data.users);
    } catch (err) {
      // ignore "we cancelled on purpose"; report the rest
      if (err instanceof DOMException && err.name === "AbortError") return;
      setError("Search failed.");
    }
  }
  search();
  return () => controller.abort(); // ❌ leaving this out = the race bug
}, [query]);
```

The function you `return` is the cleanup; React runs it before the next run and on unmount. axios accepts the same `signal`.

7. Env var mistakes (missing `VITE_`, or leaking a secret)

(a) Vite only exposes vars starting with `VITE_`; anything else is `undefined`.

```
const wrong = import.meta.env.API_URL; // ❌ undefined (no VITE_ prefix)
const baseUrl = import.meta.env.VITE_API_URL; // ✅ from VITE_API_URL in .env
```

Type it once (e.g. `vite-env.d.ts`) so `import.meta.env` isn't `any`:

```
interface ImportMetaEnv { readonly VITE_API_URL: string; }
interface ImportMeta { readonly env: ImportMetaEnv; }
```

(b) Never put a real secret in a front-end env var. Unlike server `appsettings.json`, *everything* in a React build ships to the browser — a `VITE_` value is baked into the JS, readable in DevTools. ❌

`VITE_API_KEY=secret`. ✅ Keep secrets on a backend/proxy and call *that* from React.

8. Fetching in an event handler vs. an effect

Loads **when the screen opens** → `useEffect`. Loads **because the user acted** (click, submit) → the event handler, not an effect.

```
// ❌ forcing a click action into an effect (runs on load + every keystroke):
// useEffect(() => { fetch(`/.../users/search?q=${query}`); }, [query]);

useEffect(() => { /* fetch first page on open */ }, []); // ✅ on-open
function handleSearchClick() { /* fetch for current query */ } // ✅ on-click
```

(For live "search as you type", the effect is right — match the trigger to the intent.)

9. Building the search URL by hand → breaks on spaces & symbols

A space, `+`, `&`, or `#` breaks the request. Run user text through `encodeURIComponent`.

```
const bad = `https://dummyjson.com/users/search?q=${query}`; // ❌
breaks on "John Doe"
const url = `https://dummyjson.com/users/search?q=${encodeURIComponent(query)}`; // ✅ safe
```

Bonus: `res.json()` returns `Promise<any>`, so **always annotate** it (`const data: UsersResponse = ...`) — otherwise you lose TS safety right where bad data enters.

10. Calling hooks conditionally → breaks the Rules of Hooks

Same rule as Day 4: hooks must run in the **same order every render**. Never inside `if` / loops / after an early `return`. Put the condition *inside* the effect.

```
// ❌ hook inside a condition - sometimes skipped, React breaks:
// if (userId) { useEffect(() => { /* fetch */ }, [userId]); }

// ✅ hook always runs; check goes inside
useEffect(() => {
  if (!userId) return;
  fetch(`https://dummyjson.com/users/${userId}`)
    .then((res) => res.json())
    .then((data: User) => setUser(data));
}, [userId]);
```

Quick recall

- No deps = every render; `[]` = once; `[x]` = when `x` changes.
- Never depend on the data you set inside the effect.
- Effect stays sync; define an `async` function inside and call it.
- `fetch` won't throw on 404/500 — check `res.ok`. `axios` throws on its own.

- Handle loading, error, and empty — not just the happy path.
- Cancel old requests with `AbortController` (React's `CancellationToken`).
- `VITE_` prefix required; front-end env vars are public, never secrets.
- On-load → effect, on-click → handler.
- `encodeURIComponent` your search text; always type `res.json()`.
- Never call hooks inside `if` /loops/after an early return.

4 Concept Notes — Recall & Interview (copy by pen)

Quick recall sheet for Day 5 (Hooks & API) — read before an interview to refresh the core ideas.

Effects — `useEffect` & friends

Side effect

Definition:

- Any work a component does that reaches *outside* drawing the UI — fetching data, timers, logging, changing the document title.

✗ Disadvantages / watch-outs:

- Never do side effects directly in the render body — it can run many times and cause bugs.

Example:

- `fetch('/users')` inside `useEffect` — fetching is a side effect.
- C#/.NET: like calling `HttpClient` or writing a log — work that isn't just returning a value.

`useEffect`

Definition:

- A hook that runs your side-effect code *after* the component renders.

✓ Advantages:

- Keeps fetching/timers out of render, and runs them at the right time.

✗ Disadvantages / watch-outs:

- Wrong dependencies can cause an infinite loop (render → effect → setState → render ...).

💡 Interview tip:

- Classic: "What is `useEffect` and when does it run?" → It runs *after* render, not during it.

Example:

- `useEffect(() => { load(); }, [])` — run `load` once after the first render.
- C#/.NET: loosely like a lifecycle/event hook that fires after the UI is painted.

The dependency array

Definition:

- The second argument to `useEffect` — the list of values it depends on; the effect re-runs when any of them change.

✓ Advantages:

- `[]` = run once; `[id]` = run again when `id` changes; no array = run after *every* render.

✗ Disadvantages / watch-outs:

- Missing a value used inside = stale (old) data; adding the wrong value = too many runs.

💡 Interview tip:

- Classic: "Empty array vs no array vs array with values — what's the difference?"

📄 Example:

- `useEffect(() => { load(); }, [skip])` — re-fetch when the page (skip) changes.

The cleanup function

Definition:

- The function you `return` from `useEffect`; React runs it before the next effect and when the component unmounts (leaves the screen).

✓ Advantages:

- Cancels timers, subscriptions, and in-flight requests so nothing leaks.

✗ Disadvantages / watch-outs:

- Forgetting it can leak memory or try to update a component that's already gone.

💡 Interview tip:

- Classic: "How do you clean up in `useEffect`?" → return a function from it.

📄 Example:

- `return () => clearInterval(id)` — stop the timer on cleanup.
- C#/NET: like `Dispose()` / `IDisposable` tidying up.

Effect run timing (after render)

Definition:

- Effects run *after* the browser paints the rendered screen, not while rendering.

✗ Disadvantages / watch-outs:

- Don't expect the effect to run before the user sees the first frame (measuring layout before paint is `useLayoutEffect`).

React StrictMode double-invoke (dev only)

Definition:

- In development, `<StrictMode>` runs each effect twice (setup → cleanup → setup) on purpose to help you catch missing cleanup.

✓ Advantages:

- Surfaces bugs (like a missing `AbortController`) early.

❌ Disadvantages / watch-outs:

- You may see two fetches in dev — this is normal and does NOT happen in production.

💡 Interview tip:

- "Why does my effect run twice?" → StrictMode in dev; write proper cleanup and it's fine.

Fetching data

Data-fetching lifecycle (loading / success / error)

Definition:

- A request moves through states: loading (waiting), success (got data), and error (it failed).

✅ Advantages:

- Tracking all three means the UI never looks frozen or broken.

❌ Disadvantages / watch-outs:

- Don't forget the empty state — loaded fine, but zero rows to show.

📄 Example:

- `const [status, setStatus] = useState<'loading' | 'error' | 'ready'>('loading')` — one state for the whole flow.

fetch VS axios

Definition:

- Two ways to make HTTP calls — `fetch` is built into the browser; `axios` is a small extra library.

✅ Advantages:

- `fetch`: nothing to install. `axios`: auto-parses JSON, has a base URL, interceptors, and clearer error handling.

❌ Disadvantages / watch-outs:

- `fetch` needs an extra `res.json()` step and does NOT throw on 404/500.

💡 Interview tip:

- Classic: "Difference between `fetch` and `axios`?"

📄 Example:

- `const { data } = await axios.get(url)` vs `const data = await (await fetch(url)).json()`.
- C#/.NET: both are like `HttpClient`; `axios` ~ a pre-configured `HttpClient` with handlers.

Checking HTTP status (`res.ok`)

Definition:

- With `fetch` you must check `res.ok` yourself — a 404 or 500 is NOT treated as an error automatically.

✗ Disadvantages / watch-outs:

- `axios` throws on a bad status; `fetch` quietly succeeds, so you can end up parsing an error page.

📄 Example:

- `if (!res.ok) throw new Error('HTTP ' + res.status)` — turn a bad status into a real error.

AbortController / cancelling requests

Definition:

- A browser tool to cancel a request that's still in flight.

✅ Advantages:

- Stops a finished request from updating state after the component is gone; prevents leaks.

💡 Interview tip:

- Classic: "Why cancel a request / clean up a fetch effect?"

📄 Example:

- `fetch(url, { signal })` and `return () => controller.abort()` in cleanup.
- C#/.NET: `AbortController` ~ `CancellationToken`.

Race conditions

Definition:

- When an older request finishes *after* a newer one and overwrites fresh data with stale (old) data.

✗ Disadvantages / watch-outs:

- Very common when typing fast in a search box — results arrive out of order.

📄 Example:

- Fix with `controller.abort()` on cleanup, or an `ignore` flag to drop stale results.

Config, lists & search

Environment variables (Vite `import.meta.env`)

Definition:

- Config values read from `import.meta.env`; in Vite they must start with `VITE_` to be exposed.

✗ Disadvantages / watch-outs:

- Front-end env vars are NOT secret — they get bundled and shipped to the browser.

💡 Interview tip:

- Classic: "Are front-end env vars secret?" → No; keep real secrets on the server.

Example:

- `import.meta.env.VITE_API_URL` — read the base API URL.
- C#/.NET: like `appsettings.json`, but public — never put API keys/passwords here.

Pagination (`limit` / `skip`)

Definition:

- Load data in chunks: `limit` = how many to return, `skip` = how many to skip first.

Example:

- `GET /users?limit=10&skip=20` — page 3, 10 users per page.

Search + debouncing

Definition:

- Search sends the typed query to the API; debouncing waits for a short pause in typing before firing the request.

Advantages:

- Far fewer requests and less flicker while the user types.

Example:

- `GET /users/search?q=John`; debounce with a `setTimeout` in an effect + clear it in cleanup.

Reusing logic

Custom hooks (light mention)

Definition:

- A function whose name starts with `use` that bundles hook logic so you can reuse it (e.g. `useUsers()`).

Disadvantages / watch-outs:

- Deep dive (custom hooks + reusable components) is Day 7 — today just know the idea exists.

Example:

- `const { users, loading } = useUsers()` — hide the fetch/effect logic behind one call.